

Clothes Segmentation · CVCY002

U-Net++ trained from scratch, 9-class garment segmentation

0.553

overall mIoU

0.502

foreground mIoU

0.675

mean Dice

90.7%

pixel accuracy

38

epochs trained

1 Dataset & Preprocessing

Dataset

People Clothing Segmentation (Kaggle, CC0) — 1,000 images, pixel masks, 59 original classes, uniform 825×550 resolution.

Chosen over iMaterialist (RLE masks, 24GB), Clothing Co-Parsing (.mat masks), ModaNet (polygons, no images) and DeepFashion (boxes only, no pixel masks) — this was the only option with ready-to-use PNG mask pairs.

No pretrained encoders used, per assignment constraint — both `build_unet` and `build_unet_plus_plus` train from random init.

Pipeline

1. Match image ↔ mask by id, split 80 / 10 / 10
2. Resize to 256×256 — **bilinear** for images, **nearest-neighbor** for masks (avoids invented class ids)
3. Remap 59 → 9 classes (before resizing)
4. Augment **train split only**: h-flip, crop+resize (geometric, mask-safe) · brightness/contrast/saturation (image-only)

Why remap 59 → 9 classes? A first pass with all 59 classes scored just **0.143 mIoU**, with 20+ classes stuck at exactly 0.0 IoU (ring, watch, tie, hoodie, heels...) — too few examples per class from only 1,600 training images. Grouping into 9 semantic superclasses trades garment-**type** recognition for reliable coarse-category segmentation.

Superclass	Original classes folded in
background	null
upper_clothes	blazer, blouse, bra, cape, cardigan, coat, hoodie, intimate, jacket, jumper, shirt, sweater, sweatshirt, t-shirt, top, vest
lower_clothes	jeans, leggings, panties, pants, shorts, skirt, stockings, tights
dress	bodysuit, dress, romper, swimwear
shoes	boots, clogs, flats, heels, loafers, pumps, sandals, shoes, sneakers, socks, wedges
accessories	accessories, bag, belt, bracelet, earrings, glasses, gloves, hat, necklace, purse, ring, scarf, sunglasses, tie, wallet, watch
skin	skin
hair	hair
suit	suit (kept separate — spans torso + legs, doesn't map cleanly elsewhere)

2 Model — U-Net++

Two from-scratch architectures in `model.py`: a standard U-Net baseline, and **U-Net++** (Zhou et al., 2018) — the model used for final results.

U-Net++ replaces each single skip connection with a grid of nested, densely-connected conv blocks, so the decoder at each level sees **all** prior feature maps, not just one matching encoder

Config

- Features: [32, 64, 128, 256] (4 levels, vs. paper's 5 — controls parameter count for a 1,600-image train set)
- **Deep supervision omitted** — only the final node predicts, no auxiliary losses (simpler, easier to debug; see §5 tradeoffs)

skip. This narrows the semantic gap between fine and coarse features — well suited to clothing’s fine boundaries (sleeve edges, collar lines, garment-to-garment seams).

3 Loss Function

$$\text{loss} = 0.7 \cdot \text{Dice} + 0.3 \cdot \text{weighted CE}$$

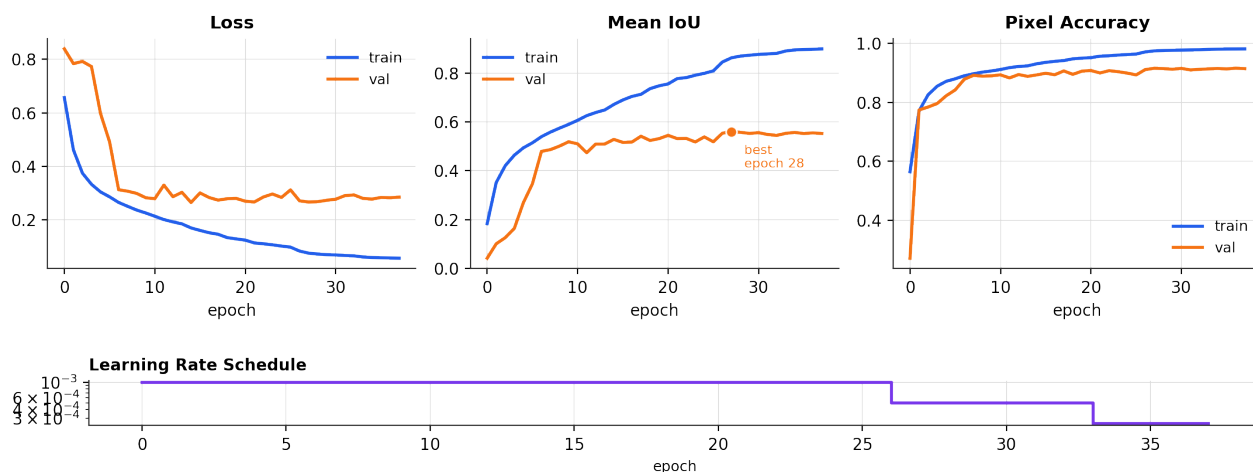
- **Dice** directly optimizes region overlap ($\approx$ IoU) — more informative than pixel accuracy under heavy class imbalance
- **Cross-entropy** (inverse-log-frequency weighted) stabilizes early gradients where Dice behaves poorly near-random

Bug found & fixed

Dice loss averaged over **all** classes, including absent ones. Since softmax never outputs exact zero, an absent class’s score was pinned to ≈ 0.32 instead of 1.0 — even for a **perfect** prediction (synthetic test: 0.655 loss, should be ≈ 0).

Fix: average only over classes present in the batch. Verified differentiable; used for all final runs.

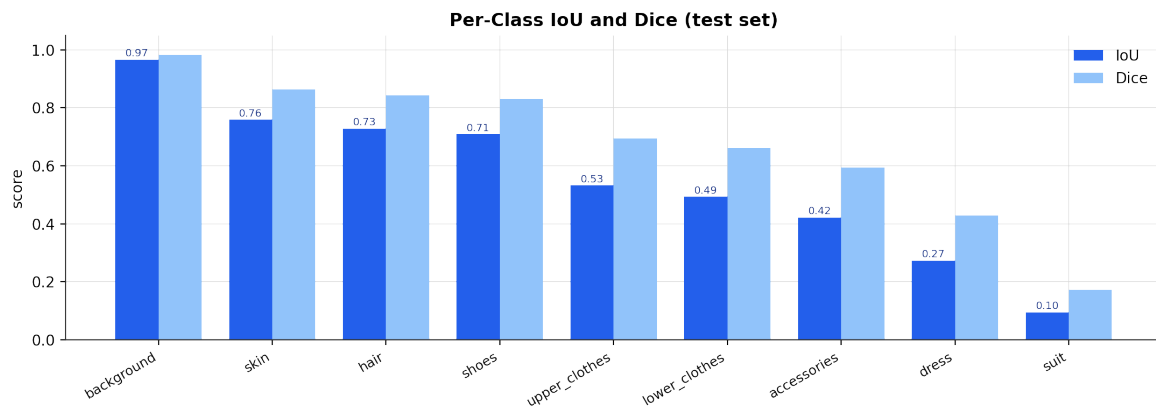
4 Training Results



Metric	Value
Best val_mean_iou (epoch 28)	0.560
val_accuracy at best checkpoint	0.910
Final-epoch train_mean_iou	0.899
Final train_loss / val_loss	0.055 / 0.284

Trained 38 of a 60-epoch budget; ModelCheckpoint monitors `val_mean_iou` and keeps epoch 28’s weights rather than the final epoch. The train/val IoU gap (0.899 vs. 0.55) by late epochs shows overfitting — expected from a small, from-scratch, no-pretrained-encoder setup.

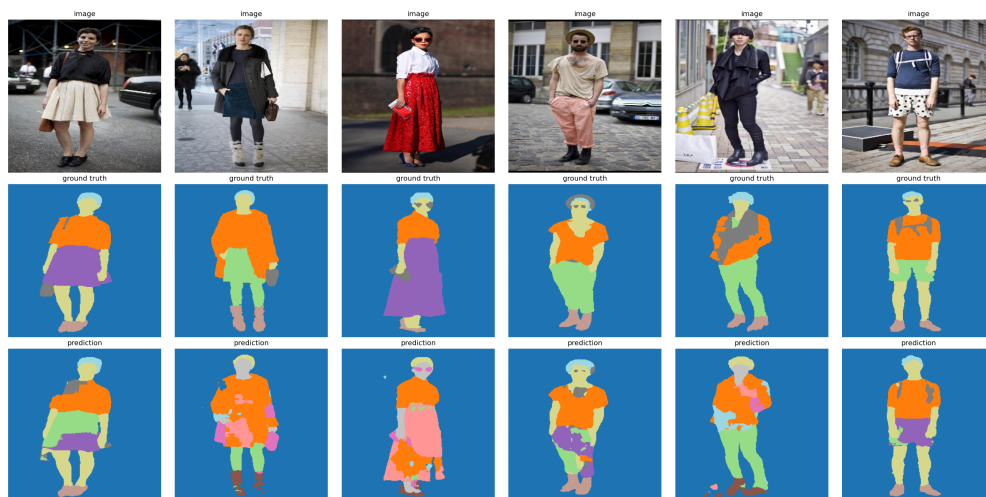
5 Test-Set Evaluation



`eval.py` reports overall mIoU, foreground mIoU (background excluded so it doesn't inflate the headline number), mean Dice, pixel accuracy, and the full per-class breakdown above, on 100 held-out test pairs.

6 Sample Predictions

Six of 100 test-set examples — input image, ground truth, and model prediction.



Predictions capture overall silhouette and pose well, but show visible noise inside garment regions (speckled misclassification between adjacent clothing classes) and softer, less precise boundaries than ground truth — consistent with the fine-boundary weaknesses discussed below.

7 Strengths, Weaknesses & Limitations

✓ Strengths

- Strong IoU on large, high-contrast regions: skin, hair, upper/lower clothes
- Robust to moderate pose & background variation (from augmentation)
- Fully self-contained — no pre-trained weights needed at inference

⚠ Weaknesses

- Coarse categories only — can't tell a t-shirt from a sweater
- Overfitting on small data: 0.899 train vs. 0.55 val mIoU
- No deep supervision (simplicity over accuracy)
- Fixed 256×256 input loses fine detail

⊖ Operating limits

- Single-person images only
- Frontal / near-frontal poses only
- Assumes clear, even lighting
- Real photographs only — not tested on illustrations/renders

Reproduce these numbers: `uv run scripts/eval.py --run-name <your-run-name> → models/<run-name>/eval/metrics.json + sample_predictions.png`